🎯 LAB 3: EXPRESS.JS SERVER MASTERY

Weeks 4-5 | Build Web Servers with Routing & Middleware

Instructor: Mohamed Amine Marzouk
Email: med.amine.mzk@horizon-tech.tn
Difficulty: ⭐⭐ Beginner-Intermediate
Time Estimate: 8-10 hours over 2 weeks
Learning Outcome: AA2 - Express.js & Web Development
Date: January 25, 2026

📍 WHERE WE ARE IN THE COURSE

You've completed **LAB 1 & LAB 2**! You now have:

- ☑ Deep async/await knowledge
- ☑ Modular code organization
- ☑ NPM package management
- ☑ Environment configuration

Now it's time to BUILD WEB SERVERS that handle thousands of requests!

🎬 WHY THIS MATTERS - THE REAL PROBLEM

The Problem: Handling Web Requests

Imagine you're trying to handle web requests manually:

```
// ❌ Handling HTTP manually - VERY complex!

const http = require("http");

const server = http.createServer((req, res) => {
  // Parse URL
  const url = req.url;
  const method = req.method;

  // Route handling (messy!)
  if (url === "/users" && method === "GET") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ users: [...] }));
  } else if (url === "/users" && method === "POST") {
    // Handle creating user
    res.writeHead(201, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ id: 1, name: "Ali" }));
  } else if (url.match(/^\/users\/\d+$/) && method === "GET") {
    // Handle getting single user
```

```
    const id = url.split("/")[2];
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ id, name: "Ali" }));
  } else if (url === "/posts" && method === "GET") {
    // Handle posts
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ posts: [...] }));
  } else {
    res.writeHead(404, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ message: "Not found" }));
  }
});

server.listen(3000);

// Problems:
// ❌ 50+ lines just to handle 5 routes
// ❌ Hard to read and maintain
// ❌ No middleware support
// ❌ No error handling
// ❌ Terrible for large applications
```

This is a NIGHTMARE! 😱

The Solution: Express.js

```
// ☑ Using Express - MUCH cleaner!

const express = require("express");
const app = express();

// Middleware
app.use(express.json());

// Routes
app.get("/users", (req, res) => {
  res.json({ users: [...] });
});

app.post("/users", (req, res) => {
  res.status(201).json({ id: 1, name: "Ali" });
});

app.get("/users/:id", (req, res) => {
  const { id } = req.params;
  res.json({ id, name: "Ali" });
});

app.get("/posts", (req, res) => {
  res.json({ posts: [...] });
});
```

```
// Error handling
app.use((err, req, res, next) => {
  res.status(500).json({ message: "Server error" });
});

app.listen(3000);

// Benefits:
// ☑ 30 lines for complete clean API
// ☑ Easy to read and maintain
// ☑ Middleware support built-in
// ☑ Error handling
// ☑ Perfect for large apps!
```

Express makes it 10x easier! ✨

🌍 REAL-WORLD COMPANIES USING EXPRESS

Company	What They Do	Why Express Matters
Uber	Ride-sharing platform	Express handles millions of API requests/second
Netflix	Video streaming	Express backend for UI requests
Facebook	Social network	Express used in internal tools & services
Google	Search giant	Express powers internal APIs
Twitter	Social media	Express used in backend infrastructure

🚀 WHAT YOU'LL BUILD IN THIS LAB

By the end, you'll create a **complete Event Management Server** with:

1. ☑ Basic Express server
2. ☑ Multiple routes (GET, POST, PUT, DELETE)
3. ☑ Route parameters and query strings
4. ☑ Middleware chains
5. ☑ Error handling
6. ☑ MVC folder structure

Live Server: Running on `localhost:3000`

Endpoints Created:

- `GET /api/events` - List all events
- `GET /api/events/:id` - Get single event
- `POST /api/events` - Create event
- `PUT /api/events/:id` - Update event

- `DELETE /api/events/:id` - Delete event
- `GET /health` - Health check

☑ PRE-LAB CHECKLIST

Before starting, verify:

- ☐ Node.js installed (`node -v`)
- ☐ npm installed (`npm -v`)
- ☐ Completed LAB 1 & LAB 2
- ☐ Understand JavaScript functions
- ☐ Understand async/await
- ☐ Understand modules (import/export)
- ☐ Code editor ready (VS Code)
- ☐ Terminal working
- ☐ 8-10 hours available
- ☐ Ready to build web servers!

📚 WHAT IS EXPRESS.JS?

Simple Definition

Express is a web framework that makes it easy to:

- Create web servers
- Handle HTTP requests
- Route different URLs
- Manage middleware
- Handle errors
- Send responses

Express in Real Life

```
USER REQUEST (Browser/Phone/App)
        ↓
INTERNET
        ↓
EXPRESS SERVER (Your Node.js app)
        ↓
ROUTING (Which endpoint?)
        ↓
MIDDLEWARE (Parse data, authenticate, etc.)
        ↓
BUSINESS LOGIC (Do something with data)
        ↓
RESPONSE (Send back data)
        ↓
BROWSER/APP DISPLAYS RESULT
```

Why Express Matters

- ☑ **Simple** - Easy to learn and use
- ☑ **Fast** - Good performance
- ☑ **Flexible** - Customize as needed
- ☑ **Popular** - Huge community
- ☑ **Production-ready** - Used by major companies
- ☑ **Middleware** - Reusable code blocks

🎯 TASK 1: CREATE YOUR FIRST EXPRESS SERVER

Step 1: Setup Project

DO THIS:

```
# Create Lab 3 folder
mkdir -p ~/node-course/lab-3
cd ~/node-course/lab-3

# Create initial files
touch package.json
touch server.js
mkdir src
```

Folder structure:

```
lab-3/
├── package.json
├── server.js
└── src/
```

Step 2: Create package.json

Edit: `package.json`

Copy this:

```
{
  "name": "lab-3-express-server",
  "version": "1.0.0",
  "description": "Learning Express.js and web servers",
  "type": "module",
```

```
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "keywords": ["express", "nodejs", "webserver"],
  "author": "Your Name",
  "license": "MIT",
  "dependencies": {
    "express": "^5.0.0"
  }
}
```

Install Express:

```
npm install
```

Wait for installation to complete!

Step 3: Create First Express Server

Create file: `server.js`

Copy this:

```
// FIRST EXPRESS SERVER
// This is your foundation for web development!

import express from "express";

const app = express();
const PORT = 3000;

// ===== MIDDLEWARE =====
// Middleware: Code that runs on EVERY request
app.use(express.json()); // Parse JSON from request body

// ===== ROUTES =====

// GET request - fetch data
app.get("/", (req, res) => {
  res.send("<h1>Welcome to Express!</h1>");
});

// Simple response
app.get("/hello", (req, res) => {
  res.json({ message: "Hello, World!" });
});
```

```
// JSON response
app.get("/api/info", (req, res) => {
  res.json({
    name: "Event Manager Server",
    version: "1.0.0",
    status: "running"
  });
});

// ===== START SERVER =====
app.listen(PORT, () => {
  console.log(`\n✅ Server is running!`);
  console.log(` 📍 Visit: http://localhost:${PORT}`);
  console.log(` 🔗 Routes:`);
  console.log(`   GET http://localhost:${PORT}/`);
  console.log(`   GET http://localhost:${PORT}/hello`);
  console.log(`   GET http://localhost:${PORT}/api/info`);
  console.log(`\n⏹  Press Ctrl+C to stop\n`);
});
```

Step 4: Run Your First Server

DO THIS:

```
npm start
```

You should see:

```
✅ Server is running!
📍 Visit: http://localhost:3000
🔗 Routes:
   GET http://localhost:3000/
   GET http://localhost:3000/hello
   GET http://localhost:3000/api/info

⏹  Press Ctrl+C to stop
```

Visit in browser:

```
http://localhost:3000
http://localhost:3000/hello
http://localhost:3000/api/info
```

🎉 **Your first server is running!**

Step 5: Understanding Express Basics

Key Concepts:

1. **app.listen(PORT)** - Start server on port
2. **app.get(path, handler)** - Handle GET requests
3. **req** - Request object (data from client)
4. **res** - Response object (send back to client)
5. **res.json()** - Send JSON response
6. **res.send()** - Send text/HTML response

☑ CHECKPOINT 1: First Server

- ☐ npm install completed
- ☐ server.js created
- ☐ Server starts without errors
- ☐ Can visit http://localhost:3000
- ☐ See all 3 routes in browser
- ☐ Understand req and res

Perfect! Move to TASK 2!

🎯 TASK 2: ROUTING & ROUTE PARAMETERS

Understanding Routes

A **route** is a path to a specific resource:

```
GET  /api/events          ← Get all events
GET  /api/events/1        ← Get event with ID 1
GET  /api/events/42       ← Get event with ID 42
POST /api/events          ← Create new event
PUT  /api/events/1        ← Update event 1
DELETE /api/events/1      ← Delete event 1
```

Step 1: Create Event Routes

Create file: `src/routes.js`

Copy this:

```
// ROUTES MODULE
// All event routes in one place
```

```
import express from "express";

const router = express.Router();

// Sample events data (in-memory)
let events = [
  {
    id: 1,
    title: "JavaScript Workshop",
    date: "2026-02-15",
    location: "Sfax",
    capacity: 30
  },
  {
    id: 2,
    title: "React Conference",
    date: "2026-03-20",
    location: "Tunis",
    capacity: 100
  },
  {
    id: 3,
    title: "Node.js Bootcamp",
    date: "2026-04-10",
    location: "Sousse",
    capacity: 25
  }
];

let nextId = 4;

// GET all events
router.get("/", (req, res) => {
  console.log("📋 GET /api/events - Fetching all events");
  res.json({
    success: true,
    count: events.length,
    data: events
  });
});

// GET single event by ID
router.get("/:id", (req, res) => {
  const { id } = req.params;
  console.log(`🔍 GET /api/events/${id}`);

  const event = events.find(e => e.id === parseInt(id));

  if (!event) {
    return res.status(404).json({
      success: false,
      message: `Event ${id} not found`
    });
```

```
    }

    res.json({
      success: true,
      data: event
    });
  });

  // POST create new event
  router.post("/", (req, res) => {
    console.log("➕ POST /api/events - Creating event");

    const { title, date, location, capacity } = req.body;

    // Validation
    if (!title || !date || !location || !capacity) {
      return res.status(400).json({
        success: false,
        message: "Missing required fields: title, date, location, capacity"
      });
    }

    // Create event
    const newEvent = {
      id: nextId++,
      title,
      date,
      location,
      capacity
    };

    events.push(newEvent);

    res.status(201).json({
      success: true,
      message: "Event created successfully",
      data: newEvent
    });
  });

  // PUT update event
  router.put("/:id", (req, res) => {
    const { id } = req.params;
    console.log(`✏️  PUT /api/events/${id} - Updating event`);

    const event = events.find(e => e.id === parseInt(id));

    if (!event) {
      return res.status(404).json({
        success: false,
        message: `Event ${id} not found`
      });
    }
```

```
    // Update fields
    const { title, date, location, capacity } = req.body;
    if (title) event.title = title;
    if (date) event.date = date;
    if (location) event.location = location;
    if (capacity) event.capacity = capacity;

    res.json({
      success: true,
      message: "Event updated successfully",
      data: event
    });
  });

  // DELETE event
  router.delete("/:id", (req, res) => {
    const { id } = req.params;
    console.log(`🗑️  DELETE /api/events/${id} - Deleting event`);

    const index = events.findIndex(e => e.id === parseInt(id));

    if (index === -1) {
      return res.status(404).json({
        success: false,
        message: `Event ${id} not found`
      });
    }

    const deletedEvent = events.splice(index, 1)[0];

    res.json({
      success: true,
      message: "Event deleted successfully",
      data: deletedEvent
    });
  });

  export default router;
```

Step 2: Update Server to Use Routes

Edit: `server.js`

Replace with this:

```
  // EXPRESS SERVER WITH ROUTING

  import express from "express";
  import eventRoutes from "./src/routes.js";
```

```
const app = express();
const PORT = 3000;

// ===== MIDDLEWARE =====
app.use(express.json());

// ===== ROUTES =====
app.get("/", (req, res) => {
  res.json({
    message: "Welcome to Event Manager API",
    endpoints: {
      events: "GET /api/events",
      createEvent: "POST /api/events",
      getEvent: "GET /api/events/:id",
      updateEvent: "PUT /api/events/:id",
      deleteEvent: "DELETE /api/events/:id",
      health: "GET /health"
    }
  });
});

// Mount event routes
app.use("/api/events", eventRoutes);

// Health check
app.get("/health", (req, res) => {
  res.json({
    status: "healthy",
    timestamp: new Date().toISOString(),
    uptime: process.uptime()
  });
});

// 404 handler
app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: `Route ${req.method} ${req.path} not found`
  });
});

// ===== START SERVER =====
app.listen(PORT, () => {
  console.log(`\n✅ Event Manager API started!`);
  console.log(` 📍 http://localhost:${PORT}`);
  console.log(`\n🔗 Available Routes:`);
  console.log(`   GET    /api/events`);
  console.log(`   POST   /api/events`);
  console.log(`   GET    /api/events/:id`);
  console.log(`   PUT    /api/events/:id`);
  console.log(`   DELETE /api/events/:id`);
  console.log(`   GET    /health`);
  console.log(`\n⏹  Press Ctrl+C to stop\n`);
});
```

Step 3: Test Your Routes

Run server:

```
npm run dev
```

Test GET all events:

```
curl http://localhost:3000/api/events
```

Test GET single event:

```
curl http://localhost:3000/api/events/1
```

Test POST create event:

```
curl -X POST http://localhost:3000/api/events \
  -H "Content-Type: application/json" \
  -d '{"title":"Docker Workshop","date":"2026-05-
10","location":"Sfax","capacity":20}'
```

Or use Postman/Insomnia (easier!)

☑ CHECKPOINT 2: Routing Works

- ☐ Server starts with routes
- ☐ GET /api/events returns all events
- ☐ GET /api/events/1 returns single event
- ☐ GET /api/events/999 returns 404
- ☐ POST creates new event
- ☐ Events array updates
- ☐ understand route parameters (:id)

🎯 TASK 3: MIDDLEWARE & REQUEST PROCESSING

Understanding Middleware

Middleware is code that runs on requests before reaching the route handler.

Flow:

```
REQUEST
  ↓
MIDDLEWARE 1 (parse JSON)
  ↓
MIDDLEWARE 2 (authenticate)
  ↓
MIDDLEWARE 3 (validate)
  ↓
ROUTE HANDLER
  ↓
RESPONSE
```

Step 1: Create Middleware File

Create file: `src/middleware.js`

Copy this:

```
// CUSTOM MIDDLEWARE

// Logger middleware - logs every request
export const logger = (req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`\n[${timestamp}] ${req.method} ${req.path}`);
  console.log(`  Body: ${JSON.stringify(req.body)}`);
  next(); // Move to next middleware
};

// Validation middleware - checks required fields
export const validateEventInput = (req, res, next) => {
  const { title, date, location, capacity } = req.body;

  // Check required fields
  if (!title || !date || !location || !capacity) {
    return res.status(400).json({
      success: false,
      message: "Missing required fields",
      required: ["title", "date", "location", "capacity"]
    });
  }

  // Validate types
  if (typeof title !== "string") {
    return res.status(400).json({
      success: false,
```

```
      message: "Title must be a string"
    });
  }

  if (typeof capacity !== "number" || capacity < 1) {
    return res.status(400).json({
      success: false,
      message: "Capacity must be a number >= 1"
    });
  }

  next();
};

// Timing middleware - measure request time
export const measureTime = (req, res, next) => {
  const start = Date.now();

  // Hook into res.json to measure when response is sent
  const originalJson = res.json;
  res.json = function(data) {
    const elapsed = Date.now() - start;
    console.log(`   ⏱️  Response time: ${elapsed}ms`);
    return originalJson.call(this, data);
  };

  next();
};

// Error handling middleware
export const errorHandler = (err, req, res, next) => {
  console.error("❌ ERROR:", err.message);

  res.status(err.status || 500).json({
    success: false,
    message: err.message || "Internal Server Error",
    error: process.env.NODE_ENV === "development" ? err : {}
  });
};
```

Step 2: Update Server to Use Middleware

Edit: `server.js`

Replace with this:

```
// EXPRESS SERVER WITH MIDDLEWARE

import express from "express";
import eventRoutes from "./src/routes.js";
```

```
import {
  logger,
  validateEventInput,
  measureTime,
  errorHandler
} from "./src/middleware.js";

const app = express();
const PORT = 3000;

// ===== GLOBAL MIDDLEWARE =====
// Order matters! These run on EVERY request

app.use(express.json()); // Parse JSON
app.use(logger);         // Log requests
app.use(measureTime);    // Measure time

// ===== ROUTES =====

app.get("/", (req, res) => {
  res.json({
    message: "Welcome to Event Manager API",
    endpoints: {
      events: "GET /api/events",
      createEvent: "POST /api/events",
      getEvent: "GET /api/events/:id",
      updateEvent: "PUT /api/events/:id",
      deleteEvent: "DELETE /api/events/:id"
    }
  });
});

// Mount event routes
// Apply validation ONLY to POST and PUT
app.use("/api/events", (req, res, next) => {
  if ((req.method === "POST" || req.method === "PUT") && req.path === "/") {
    validateEventInput(req, res, next);
  } else {
    next();
  }
});

app.use("/api/events", eventRoutes);

// Health check
app.get("/health", (req, res) => {
  res.json({
    status: "healthy",
    timestamp: new Date().toISOString()
  });
});

// 404 handler (must be last)
app.use((req, res) => {
```

```
  res.status(404).json({
    success: false,
    message: `Route not found: ${req.method} ${req.path}`
  });
});

// Error handler (must be last)
app.use(errorHandler);

// ===== START SERVER =====
app.listen(PORT, () => {
  console.log(`\n✅ Event Manager API with Middleware!`);
  console.log(` 📍 http://localhost:${PORT}\n`);
});
```

Step 3: Test Middleware

Run server:

```
npm run dev
```

Test with invalid data (missing fields):

```
curl -X POST http://localhost:3000/api/events \
  -H "Content-Type: application/json" \
  -d '{"title":"Workshop"}'
```

You'll see:

- Request logged
- Validation error
- Response time measured

✅ CHECKPOINT 3: Middleware Works

- ☐ Logger middleware logs requests
- ☐ Validation middleware checks fields
- ☐ Time measurement works
- ☐ Error responses are formatted
- ☐ Middleware chain works

🎯 TASK 4: MVC FOLDER STRUCTURE

Understanding MVC

MVC = Model, View, Controller

```
MODEL       ← Data & database operations
   ↓
CONTROLLER  ← Business logic
   ↓
VIEW        ← Response to client
```

Step 1: Create MVC Structure

DO THIS:

```
# Create folders
mkdir src/models
mkdir src/controllers
mkdir src/routes

# Move routes to controllers
touch src/controllers/eventController.js
touch src/models/Event.js
touch src/routes/eventRoutes.js
```

New structure:

```
lab-3/
├── src/
│   ├── models/
│   │   └── Event.js
│   ├── controllers/
│   │   └── eventController.js
│   ├── routes/
│   │   └── eventRoutes.js
│   ├── middleware.js
│   └── routes.js (old - we'll replace)
├── server.js
└── package.json
```

Step 2: Create Model

Create file: `src/models/Event.js`

Copy this:

```
// EVENT MODEL
// Represents event data and operations

// In-memory database
let events = [
  {
    id: 1,
    title: "JavaScript Workshop",
    date: "2026-02-15",
    location: "Sfax",
    capacity: 30
  },
  {
    id: 2,
    title: "React Conference",
    date: "2026-03-20",
    location: "Tunis",
    capacity: 100
  }
];

let nextId = 3;

export class Event {
  static getAll() {
    return events;
  }

  static getById(id) {
    return events.find(e => e.id === parseInt(id));
  }

  static create(data) {
    const newEvent = {
      id: nextId++,
      ...data,
      createdAt: new Date().toISOString()
    };
    events.push(newEvent);
    return newEvent;
  }

  static update(id, data) {
    const event = this.getById(id);
    if (!event) return null;

    Object.assign(event, data);
    event.updatedAt = new Date().toISOString();
    return event;
  }

  static delete(id) {
    const index = events.findIndex(e => e.id === parseInt(id));
```

```
    if (index === -1) return null;

    return events.splice(index, 1)[0];
  }
}

export default Event;
```

Step 3: Create Controller

Create file: `src/controllers/eventController.js`

Copy this:

```
// EVENT CONTROLLER
// Handles business logic and routing

import Event from "../models/Event.js";

export class EventController {
  // GET all events
  static getAllEvents(req, res) {
    try {
      const events = Event.getAll();
      res.json({
        success: true,
        count: events.length,
        data: events
      });
    } catch (error) {
      res.status(500).json({
        success: false,
        message: error.message
      });
    }
  }

  // GET single event
  static getEventById(req, res) {
    try {
      const { id } = req.params;
      const event = Event.getById(id);

      if (!event) {
        return res.status(404).json({
          success: false,
          message: `Event ${id} not found`
        });
      }
```

```
      res.json({
        success: true,
        data: event
      });
    } catch (error) {
      res.status(500).json({
        success: false,
        message: error.message
      });
    }
  }

  // CREATE new event
  static createEvent(req, res) {
    try {
      const { title, date, location, capacity } = req.body;

      const newEvent = Event.create({
        title,
        date,
        location,
        capacity
      });

      res.status(201).json({
        success: true,
        message: "Event created successfully",
        data: newEvent
      });
    } catch (error) {
      res.status(400).json({
        success: false,
        message: error.message
      });
    }
  }

  // UPDATE event
  static updateEvent(req, res) {
    try {
      const { id } = req.params;
      const updatedEvent = Event.update(id, req.body);

      if (!updatedEvent) {
        return res.status(404).json({
          success: false,
          message: `Event ${id} not found`
        });
      }

      res.json({
        success: true,
        message: "Event updated successfully",
        data: updatedEvent
```

```
      });
    } catch (error) {
      res.status(400).json({
        success: false,
        message: error.message
      });
    }
  }

  // DELETE event
  static deleteEvent(req, res) {
    try {
      const { id } = req.params;
      const deletedEvent = Event.delete(id);

      if (!deletedEvent) {
        return res.status(404).json({
          success: false,
          message: `Event ${id} not found`
        });
      }

      res.json({
        success: true,
        message: "Event deleted successfully",
        data: deletedEvent
      });
    } catch (error) {
      res.status(500).json({
        success: false,
        message: error.message
      });
    }
  }
}

export default EventController;
```

Step 4: Create Routes

Create file: `src/routes/eventRoutes.js`

Copy this:

```
// EVENT ROUTES
// Define all event endpoints

import express from "express";
import EventController from "../controllers/eventController.js";
```

```
const router = express.Router();

// Routes
router.get("/", EventController.getAllEvents);
router.get("/:id", EventController.getEventById);
router.post("/", EventController.createEvent);
router.put("/:id", EventController.updateEvent);
router.delete("/:id", EventController.deleteEvent);

export default router;
```

Step 5: Update Server

Edit: `server.js`

Replace with this:

```
// EXPRESS SERVER WITH MVC STRUCTURE

import express from "express";
import eventRoutes from "./src/routes/eventRoutes.js";
import {
  logger,
  validateEventInput,
  measureTime
} from "./src/middleware.js";

const app = express();
const PORT = 3000;

// ===== MIDDLEWARE =====
app.use(express.json());
app.use(logger);
app.use(measureTime);

// ===== ROUTES =====

app.get("/", (req, res) => {
  res.json({
    message: "🎉 Event Manager API (MVC Structure)",
    version: "1.0.0",
    endpoints: {
      getAllEvents: "GET /api/events",
      getEvent: "GET /api/events/:id",
      createEvent: "POST /api/events",
      updateEvent: "PUT /api/events/:id",
      deleteEvent: "DELETE /api/events/:id",
      health: "GET /health"
    }
  });
```

```
});

// Validation middleware for POST/PUT
const validateEvent = (req, res, next) => {
  if (req.method === "POST" || req.method === "PUT") {
    validateEventInput(req, res, next);
  } else {
    next();
  }
};

app.use("/api/events", validateEvent);
app.use("/api/events", eventRoutes);

// Health check
app.get("/health", (req, res) => {
  res.json({
    status: "✅ healthy",
    uptime: process.uptime().toFixed(2) + "s"
  });
});

// 404 handler
app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: `Not found: ${req.method} ${req.path}`
  });
});

// ===== START SERVER =====
app.listen(PORT, () => {
  console.log(`\n✅ Event Manager API (MVC) started!`);
  console.log(` 📍 http://localhost:${PORT}`);
  console.log(`\n🏗️  Architecture: Model-View-Controller`);
  console.log(`   Models/   → Data operations`);
  console.log(`   Controllers/ → Business logic`);
  console.log(`   Routes/    → API endpoints\n`);
});
```

Step 6: Clean Up Old Files

Remove:

```
rm src/routes.js  # Old file - we replaced it
```

✅ CHECKPOINT 4: MVC Structure

- ☐ Models folder created
- ☐ Controllers folder created
- ☐ Routes folder created
- ☐ Event model works
- ☐ EventController handles logic
- ☐ Routes connected properly
- ☐ Server starts successfully
- ☐ All endpoints work

🎯 TASK 5: COMPLETE WORKING API

Final Integration

Let's make a complete, professional API!

Step 1: Add Response Helper

Create file: `src/utils/responses.js`

Copy this:

```
// RESPONSE HELPER
// Standardized response formats

export class ApiResponse {
  static success(data, message = "Success", statusCode = 200) {
    return {
      success: true,
      message,
      data,
      timestamp: new Date().toISOString()
    };
  }

  static error(message = "Error", statusCode = 400, data = null) {
    return {
      success: false,
      message,
      data,
      timestamp: new Date().toISOString()
    };
  }

  static paginated(data, page, limit, total) {
    return {
      success: true,
      data,
      pagination: {
```

```
          page,
          limit,
          total,
          pages: Math.ceil(total / limit)
        }
      };
    }
  }
```

Step 2: Enhanced Demo

Create file: `test-api.js`

Copy this:

```
// TEST API ENDPOINTS
// Run: node test-api.js

import http from "http";

// Helper function to make requests
function makeRequest(method, path, data = null) {
  return new Promise((resolve, reject) => {
    const options = {
      hostname: "localhost",
      port: 3000,
      path: path,
      method: method,
      headers: {
        "Content-Type": "application/json"
      }
    };

    const req = http.request(options, (res) => {
      let body = "";
      res.on("data", (chunk) => body += chunk);
      res.on("end", () => {
        resolve({
          status: res.statusCode,
          body: JSON.parse(body)
        });
      });
    });

    req.on("error", reject);

    if (data) {
      req.write(JSON.stringify(data));
    }
    req.end();
```

```
  });
}

// Test all endpoints
async function testAPI() {
  console.log("═══════════════════════════════════════");
  console.log("   TESTING EVENT MANAGER API");
  console.log("═══════════════════════════════════════\n");

  try {
    // Test 1: GET all events
    console.log("1️⃣  GET /api/events\n");
    let result = await makeRequest("GET", "/api/events");
    console.log("Status:", result.status);
    console.log("Events:", result.body.count, "\n");

    // Test 2: CREATE event
    console.log("2️⃣  POST /api/events\n");
    result = await makeRequest("POST", "/api/events", {
      title: "Docker Masterclass",
      date: "2026-06-01",
      location: "Sfax",
      capacity: 35
    });
    console.log("Status:", result.status);
    console.log("Created:", result.body.data.title, "\n");

    // Test 3: GET single event
    console.log("3️⃣  GET /api/events/1\n");
    result = await makeRequest("GET", "/api/events/1");
    console.log("Status:", result.status);
    console.log("Event:", result.body.data.title, "\n");

    // Test 4: UPDATE event
    console.log("4️⃣  PUT /api/events/1\n");
    result = await makeRequest("PUT", "/api/events/1", {
      title: "Advanced JavaScript Workshop"
    });
    console.log("Status:", result.status);
    console.log("Updated:", result.body.data.title, "\n");

    // Test 5: DELETE event
    console.log("5️⃣  DELETE /api/events/2\n");
    result = await makeRequest("DELETE", "/api/events/2");
    console.log("Status:", result.status);
    console.log("Deleted:", result.body.data.title, "\n");

    // Test 6: GET all (verify delete)
    console.log("6️⃣  GET /api/events (after delete)\n");
    result = await makeRequest("GET", "/api/events");
    console.log("Status:", result.status);
    console.log("Remaining events:", result.body.count, "\n");

    // Test 7: Error handling (invalid data)
```

```
    console.log("7️⃣  POST /api/events (invalid)\n");
    result = await makeRequest("POST", "/api/events", {
      title: "Incomplete Event"
      // Missing: date, location, capacity
    });
    console.log("Status:", result.status);
    console.log("Error:", result.body.message, "\n");

    // Test 8: 404 error
    console.log("8️⃣  GET /api/events/999\n");
    result = await makeRequest("GET", "/api/events/999");
    console.log("Status:", result.status);
    console.log("Message:", result.body.message, "\n");

    console.log("═══════════════════════════════════════");
    console.log("   ✅ ALL TESTS COMPLETE!");
    console.log("═══════════════════════════════════════");

  } catch (error) {
    console.error("❌ Test failed:", error.message);
  }
}

// Run tests
console.log("Waiting for server...\n");
setTimeout(testAPI, 1000);
```

Step 3: Run Complete Demo

Terminal 1 (Start server):

```
npm start
```

Terminal 2 (Run tests):

```
node test-api.js
```

You'll see:

```
═══════════════════════════════════════
   TESTING EVENT MANAGER API
═══════════════════════════════════════

1️⃣  GET /api/events

Status: 200
```

```
Events: 2

2️⃣  POST /api/events

Status: 201
Created: Docker Masterclass

... (more tests)

✅ ALL TESTS COMPLETE!
```

✅ CHECKPOINT 5: Complete API

- ☐ All CRUD operations work
- ☐ Validation works
- ☐ Error handling works
- ☐ MVC structure organized
- ☐ Middleware chain works
- ☐ Tests pass successfully

🎁 BONUS CHALLENGES

Bonus 1: Query String Filtering

Add filtering to GET /api/events:

```
GET /api/events?location=Sfax
GET /api/events?capacity=30
GET /api/events?location=Sfax&capacity=30
```

Bonus 2: Sorting

Add sorting to GET /api/events:

```
GET /api/events?sort=date
GET /api/events?sort=-capacity
```

Bonus 3: Pagination

Add pagination:

```
GET /api/events?page=1&limit=5
```

Bonus 4: Search

Add search by title:

```
GET /api/events?search=javascript
```

Bonus 5: Error Classes

Create custom error classes for better error handling

🆘 TROUBLESHOOTING

Error: "Cannot find module express"

Solution: Run `npm install`

Port 3000 already in use

Solution: Change PORT in server.js or kill process:

```
lsof -i :3000
kill -9 <PID>
```

Route not working

Solution: Check route order in server.js - specifics before generics!

Middleware not running

Solution: Middleware order matters! Global middleware before routes!

JSON parsing error

Solution: Make sure to send `Content-Type: application/json` header

📚 LAB 3 DELIVERABLES

Submit these files:

1. ☑ `server.js` - Main Express server
2. ☑ `src/models/Event.js` - Event model
3. ☑ `src/controllers/eventController.js` - Business logic
4. ☑ `src/routes/eventRoutes.js` - Routes definition

5. ☑ `src/middleware.js` - Custom middleware
6. ☑ `test-api.js` - API tests
7. ☑ `package.json` - Dependencies
8. ☑ `README.md` - How to run

🎓 GRADING RUBRIC

Criterion	Points
Server Runs	15
Routes Work (CRUD)	20
MVC Structure	20
Middleware	15
Error Handling	15
Code Quality	10
TOTAL	**100**

🎉 CONGRATULATIONS!

You've completed **LAB 3: Express.js Servers**!

You now can: ☑ Create web servers with Express
☑ Handle HTTP requests
☑ Implement CRUD operations
☑ Use middleware
☑ Organize with MVC
☑ Build production-ready APIs

Ready for LAB 4: REST API Design! 🚀

Instructor: Mohamed Amine Marzouk
Institution: ISITCom - PRINCE Laboratory
Email: med.amine.mzk@horizon-tech.tn
Last Updated: January 25, 2026
Status: ☑ Ready for Students